Low-Level Design (LLD)

1. **Module-Level Design**

---

1. Document Processing Module (loader.py)

- Function: load_pdf(path)
- Input: PDF file path
- Output: List of documents

```
documents = loader.load()
```

2. Chunking Module (chunker.py)

- Function: chunk_data(documents)
- Input: Documents
- Output: Chunks

```
chunks = splitter.split_documents(documents)
```

3. Embedding Module (embeddings.py)

- Function: create_vector_store(chunks)
- Converts chunks → embeddings
- Stores in ChromaDB

4. Vector Storage Module

- Tool: ChromaDB
- Stores:
    - chunk text
    - embeddings

    - metadata

5. Retrieval Module (retriever.py)

- Function: get_retriever(vectordb)
- Uses similarity search
- Returns top-k chunks

6. Query Processing Module (rag_pipeline.py)

- Function: generate_answer(query, retriever, llm)
- Steps:
    1. Retrieve chunks
    2. Create context
    3. Send to LLM
    4. Return answer

7. Graph Execution Module (langgraph_flow.py)

- Controls workflow
- Nodes:
    - process_node
    - output_node

8. HITL Module

- Trigger conditions:
    - Low answer quality
    - Missing context
- Accepts human input

2. Data Structures

Document Object

```
{
   "page_content": "text",
   "metadata": {"page": 1}
}
```

Chunk Object

```
{
   "page_content": "chunk text",
   "metadata": {"chunk_id": 1}
}
```

Embedding Structure

```
[0.12, 0.45, 0.67, ...]
```

Query-Response Schema

```
{
   "query": "What is ML?",
   "answer": "Machine learning is..."
}
```

Graph State Object

```
class State(TypedDict):
   query: str
   pipeline: object
   answer: str
```

---

3. Workflow Design (LangGraph)

Nodes

1. Process Node

- Takes query
- Calls RAG pipeline
- Generates answer

2. Output Node

- Displays answer

Edges

Process → Output

Flow

Input → Process Node → Output Node

---

4. Conditional Routing Logic

Normal Flow

If answer is valid:

Process → Output

Escalation Flow

Trigger when:

- Answer contains “don’t know”

- Answer too short
- No relevant chunks

Process → HITL → Output

---

5. HITL Design

When Triggered

- Low confidence answer
- Missing context

Flow

System → Detect issue → Ask human → Replace answer

Human Input

human = input("Enter human response:")

Final Output

Human answer is returned to user

---

6. API / Interface Design

Input

User enters query via CLI

Output

Answer printed in terminal

Interaction Flow

User → Input → Graph → Answer → Display

---

7. **Error Handling**

Missing PDF

- Raise file error

No Chunks Found

- Return fallback message

LLM Failure

- Catch exception
- Show error message

API Errors

- Handle invalid model / key

- Maintainability
- Scalability
- Debuggability